[Khanh Pham] IPC: Pipe, Message queue, Shared memory, Signal

IPC

We know process have separate memory space address. In this chapter, we will learn about the way processes can communicate together.

Pipe

Pipe is the IPC that processes create and use kernel buffer for communicating.

Pipes can be used only between processes that have a common ancestor.

Normally, a pipe is created by a process, that process calls fork, and the pipe is used between the parent and the child.

We can create pipe by using pipe function:

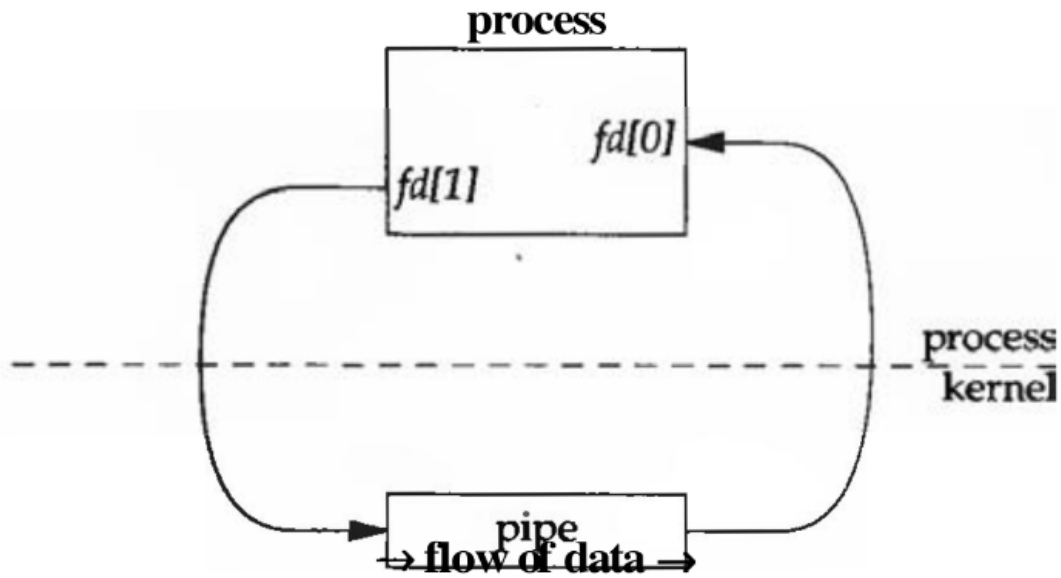
```
int pipe(int fd[2])
```

Two file descriptors are returned: `fd[0]`, which is open for reading, and `fd[1]`, which is open for writing.

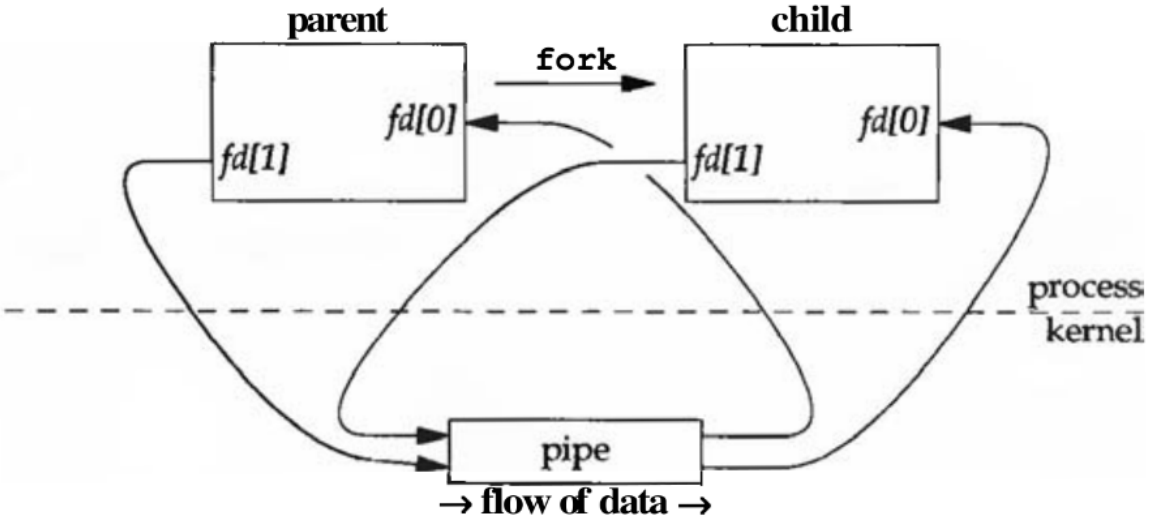
When all processes close the file descriptors which point to the pipe buffer, the kernel will discard the buffer.

We can imagine the way to use pipe in bellow:

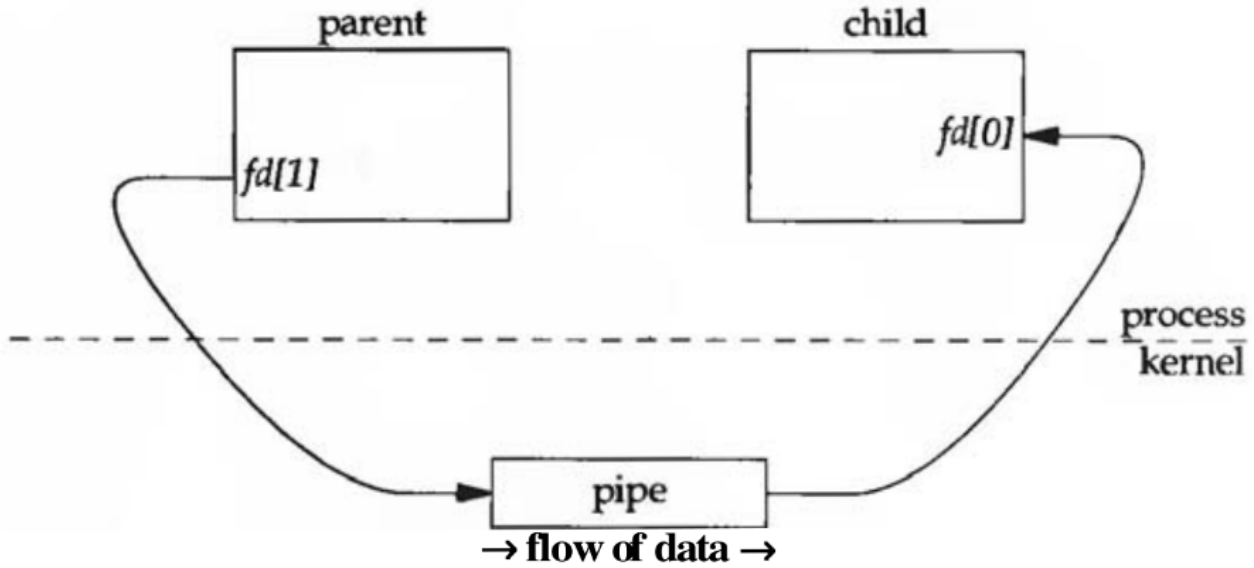
- A process create pipe



- This process use fork function to create child process:



- Next, the parent process closes the read end of one pipe, and the child process closes the write end of that same pipe



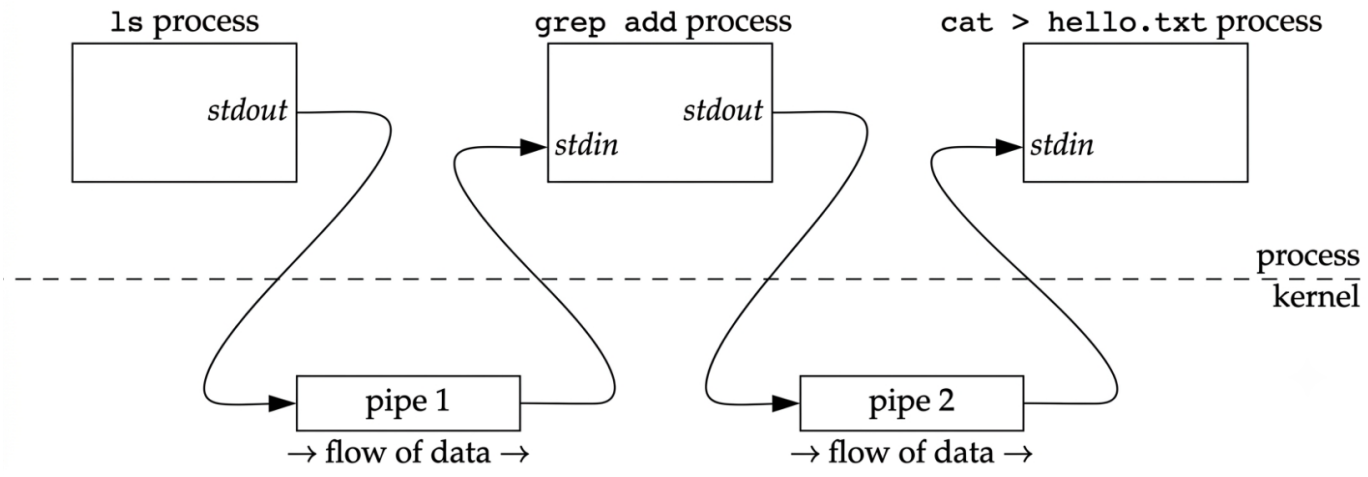
When we use the | (pipe) operator in the shell, the output of one command is passed as the input to the next command.

For example, when we run:

```
ls | grep add | cat > test.txt
```

the output of `ls` is passed to `grep add`, and the output of `grep add` is then passed to `cat`. Finally, the output of `cat` is redirected to `test.txt`.

In other words, the | operator allows commands to communicate by passing the output of one command to the input of the next command.



Therefore, the result in `test.txt` when I run in my device is:

```
add.c
add.i
add.o
```

```
libadd.a
libadd.so
```

In pipe, we use `read` systemcall to get the data from pipe and `write` to push data to pipe.

> From linux 2.6.11, the pipe capacity is 16 page, that mean 16 x 4096 = 65536(bytes)

We can use the function `fcntl` with the flags: `F_SETPIPE_SZ` and `F_GETPIPE_SZ` to set the maximum of pipe capacity. The pipe capacity must is power-of-two page-size multiple of the requested size

For example:

```
    int fd[2];
    pipe(fd);
    char temp = 'h';
    int count = 0;

    while (1)
    {
        write(fd[1], &temp, 1);
        printf("%d byte\n", ++count);
    }
```

The maximum pipe capacity is:

```
65536 byte
```

When we use fcntl to set the pipe size:

```
int size = 4096 * 129;
int main()
int fd[2];
    pipe(fd);
    char temp = 'h';
    int count = 0;
    fcntl(fd[1], F_SETPIPE_SZ, size);

    while (1)
    {
```

```
        write(fd[1], &temp, 1);
        printf("%d byte\n", ++count);
    }
```

The result is:

```
1048576 byte
```

The result is 256 * 4096 (bytes)?

Why we need power of two number of the page size in pipe capacity?

> “
>
> > “ According to the comment of torvalds linux source comment https://github.com/torvalds/linux/blob/master/fs/pipe.c, pipe implemented like ring buffer, that mean it has tail and head to manage(tail for reading, head for writing). When the number of page size in pipe is power-of-2 like 128, the head can easily return to first offset (0) by the `head & (128 - 1)`

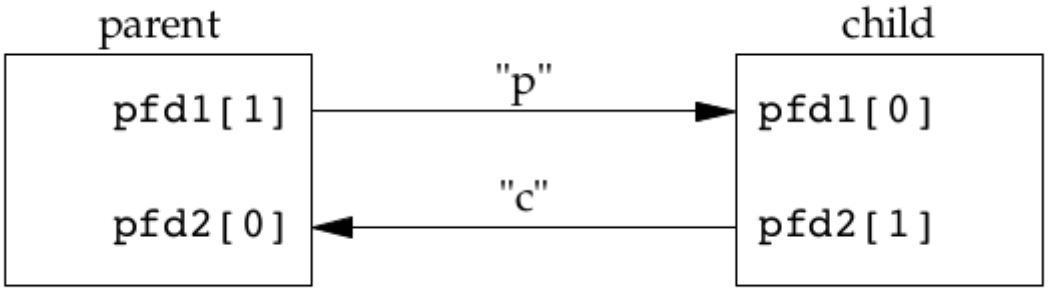
Why the page size always are the power of 2?

> “
>
> > “ When we want to get the page number and offset of every address in memory, we need use the operator: divide `/` for page number and MOD `%` for page offset. For example: the page size is 32 and the address we need to calculate is 104. The page number is `104 / 32 = 3`, the offset is `104 % 32 = 8`

> “
>
> > “ When the page size is power of 2, like 32 in this example, we can get the offset by the value of n(n is the number to present the page size in binary, 5 in this case) and the page number by the value of address after right shift n times. As the example above, the page size is 32, the address is 104 =

> 0110 0100, the offset is the last 5 bit(00100) = 8, the page number is 0110 0100 >> 5 = 0000 0011 = 3

> If we want to use pipe for full-duplex communication, we need use 2 pipes.



Read and write behavior

However, the `read` and `write` function for pipe isn't simple, they have different behavior that I list in two tables bellow

The read behavior in block and non-block state

State	Situation	Read return	The program status
Blocking	Writing process write and read process read n bytes	n	Running
Blocking	No process open for writing	0	Running
Blocking	Some process open for writing and the pipe is empty		Block until writing process write
Non-block	Some process open for writing	-1	Running

The read behavior in block and non-block state

State	Situation	Write return	The program status
Blocking	Writing process write n bytes and the pipe isn't full	n	Running
Blocking	The pipe is full		Block

State	Situation	Write return	The program status
Blocking	No process open for reading	n	Write until the pipe full
Non-block	The pipe is full	-1	Running

> Therefore, when use read and write in pipe, we need to be careful the deadlock situation like both 2 process block because of calling read before write syscal when using 2 pipes:

```
int main()
{

    pid_t pid;
    int fd1[2];
    int fd2[2];
    pipe(fd1);
    pipe(fd2);

    pid = fork();
    if (pid == 0)
    {
        char c;
        close(fd1[0]);
        close(fd2[1]);
        read(fd2[0], &c, 1);
    }
    else
    {
        char c;
        close(fd2[0]);
        close(fd1[1]);
        read(fd1[0], &c, 1);
    }
}
```

After understanding the pipe IPC, we need the IPC to communicate between normal processes, that mean processes without the ancestor relationship. We can use Named pipe(FIFO) that will be discussed bellow:

FIFO

Don't like the unamed pipe: we can't use the pathname to open the fd, FIFO create the buffer in kernel that have the specific pathname. As a result, all process can use this pathname to open and use the pipe.

```
/* process 1*/
int main(void)
{
    char c[5];

    mkfifo("/tmp/myfifo", 0666);

    int fd = open("/tmp/myfifo", O_WRONLY);

    while (fgets(c, sizeof(c), stdin))
    {
        write(fd, c, strlen(c));
        sleep(2);
    }

    close(fd);
    return 0;
}

/* process 2*/
int main(void)
{
    char c[5];

    int fd = open("/tmp/myfifo", O_RDONLY);

    while (1)
    {
        int n = read(fd, c, sizeof(c) - 1);

        c[n] = '\0';

        printf("%s", c);
```

```
        fflush(stdout);
    }

    close(fd);
    return 0;
}
```

In this example, I use 2 process, 1 for writing, 1 for reading. Any character from stdin of process 1 will be push into the FIFO. After that, the process 2 can get this character from FIFO buffer.

The open syscal have some behavior: \

Open behavior

Type of open()	Flags	Other end of FIFO open	Other end of FIFO closed
Reading	Blocking	Sucess	Block
Reading	Non-Blocking	Sucess	Sucess
Writing	Blocking	Sucess	Block
Writing	Non-Blocking	Sucess	Fail

> When we close all fd reference to the FIFO, the buffer in kernel will be disappeared, however, the FIFO pathname still exist in file system. If we want to delete it too, we must use `unlink` system call

Pipe and FIFO have two disadvantages.

- Firstly, they are process-persistent, which means that the buffer storing data only exists as long as at least one process keeps the file descriptor (fd) of this buffer open. When no process has the fd of this buffer open, the kernel will automatically release the buffer.
- Secondly, they transfer data byte by byte. This means that if the system has more than one sender, the receiver needs to split the data from the buffer when each sender process sends data with a different length.

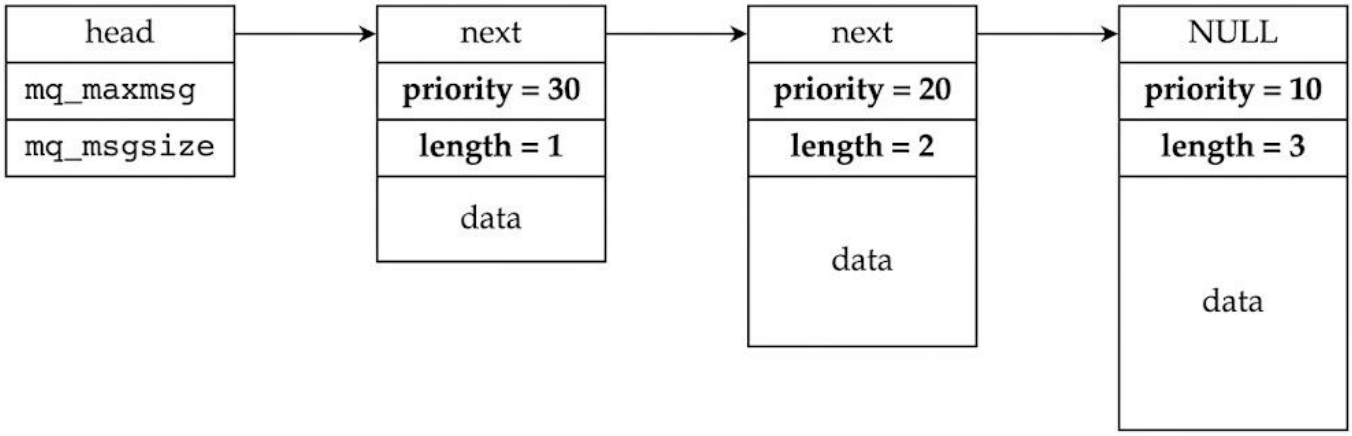
The IPC mechanism below will avoid these problems.

POSIX Message queue

The message queue(MQ) have pathname in file system like FIFO, but it have more specific features:

- MQ is kernel-persistent, that means this buffer only is released when the kernel reboot or until the object is explicitly deleted.
- MQ is not only tranfer data like PIPE of FIFO, each sending process when to push data into MQ need pack this data to a message with 3 type: priority, length and data. Inside the queue, the highest priority message always stay in the first place, so when the receive pop message, it always get the highest priority message.

We can imagine MQ like a linked list:



This table list all api of POSIX MQ:

API	Detail
mq_open()	Create a new message queue or opens an existing message queue
mq_close()	Close the message queue descriptor
mq_unlink()	Remove the message queue identified by name, and marks the queue to be destroyed
mq_send()	Add the message in the buffer
mq_receive()	Remove the oldest message with the highest priority from the message queue
mq_notify()	Register the calling process to receive a notification when a message arrives on the empty queue
mq_getattr()	Get the attribute of MQ
mq_setattr()	Set the attribute of MQ

For example:

```
/* Sender */
#define QUEUE_NAME "/prio_queue"

void send_msg(mqd_t mq, const char *text, unsigned int prio)
{
    if (mq_send(mq, text, strlen(text) + 1, prio) == 0)
    {
        printf("Send %s, priority = %u\n", text, prio);
    }
}

int main()
{
    mqd_t mq = mq_open(QUEUE_NAME, O_CREAT | O_WRONLY, 0666, NULL);
    if (mq == (mqd_t)-1)
    {
        perror("mq_open");
        exit(1);
    }

    send_msg(mq, "mission 1(normal)", 1);
    send_msg(mq, "mission 2(normal)", 1);
    send_msg(mq, "mission 3(important)", 10);
    send_msg(mq, "mission 4(normal)", 1);
    send_msg(mq, "mission 5(quite important)", 5);
    send_msg(mq, "mission 6(normal)", 1);

    mq_close(mq);
    return 0;
}

/* Receiver */
#define QUEUE_NAME "/prio_queue"
#define MAX_SIZE 8192

int main()
{
    mqd_t mq = mq_open(QUEUE_NAME, O_RDONLY);
    if (mq == (mqd_t)-1)
    {
```

```
        perror("Don't have the MQ");
        exit(1);
    }

    char buffer[MAX_SIZE];
    unsigned int prio;

    while (1)
    {
        if (mq_receive(mq, buffer, MAX_SIZE, &prio) != -1)
        {
            printf("Receive %s, priority = %u\n", buffer, prio);
        }
    }

    mq_close(mq);
    mq_unlink(QUEUE_NAME);
    return 0;
}
```

When we run the sender first, the receive result is:

```
Receive mission 3(important), priority = 10
Receive mission 5(quite important), priority = 5
Receive mission 1(normal), priority = 1
Receive mission 2(normal), priority = 1
Receive mission 4(normal), priority = 1
Receive mission 6(normal), priority = 1
```

So we can see, the message with the highest priority always stay in the first position in queue.

> The send and receive function in MQ is atomic operation. Accoring to https://elixir.bootlin.com/linux/v7.2.4/source/ipc/mqueue.c, this functions implement the atomic operation by using spinlock.

MQ also have Blocking and Non-Blocking state in `mq_send` and `mq_receive` function.

Now, after understanding some of data transfer IPC(PIPE, FIFO and MQ), we see the problem is this IPC call System call too much, so we need other IPC to avoid calling systemcall in working time. We can do that by shared memory IPC that will be discussed bellow:

Before learning about POSIX Shared memory, we need understand the memory mapping(mmap) system call first.

Memory mapping(mmap)

The mmap() system call creates a new memory mapping in the calling process’s virtual address space

There are 2 ways of mapping:

- **File mapping**: Using a file in filesystem to access the mapping memory.
- **Anonymous mapping**: Using directly virtual address to access the mapping memory

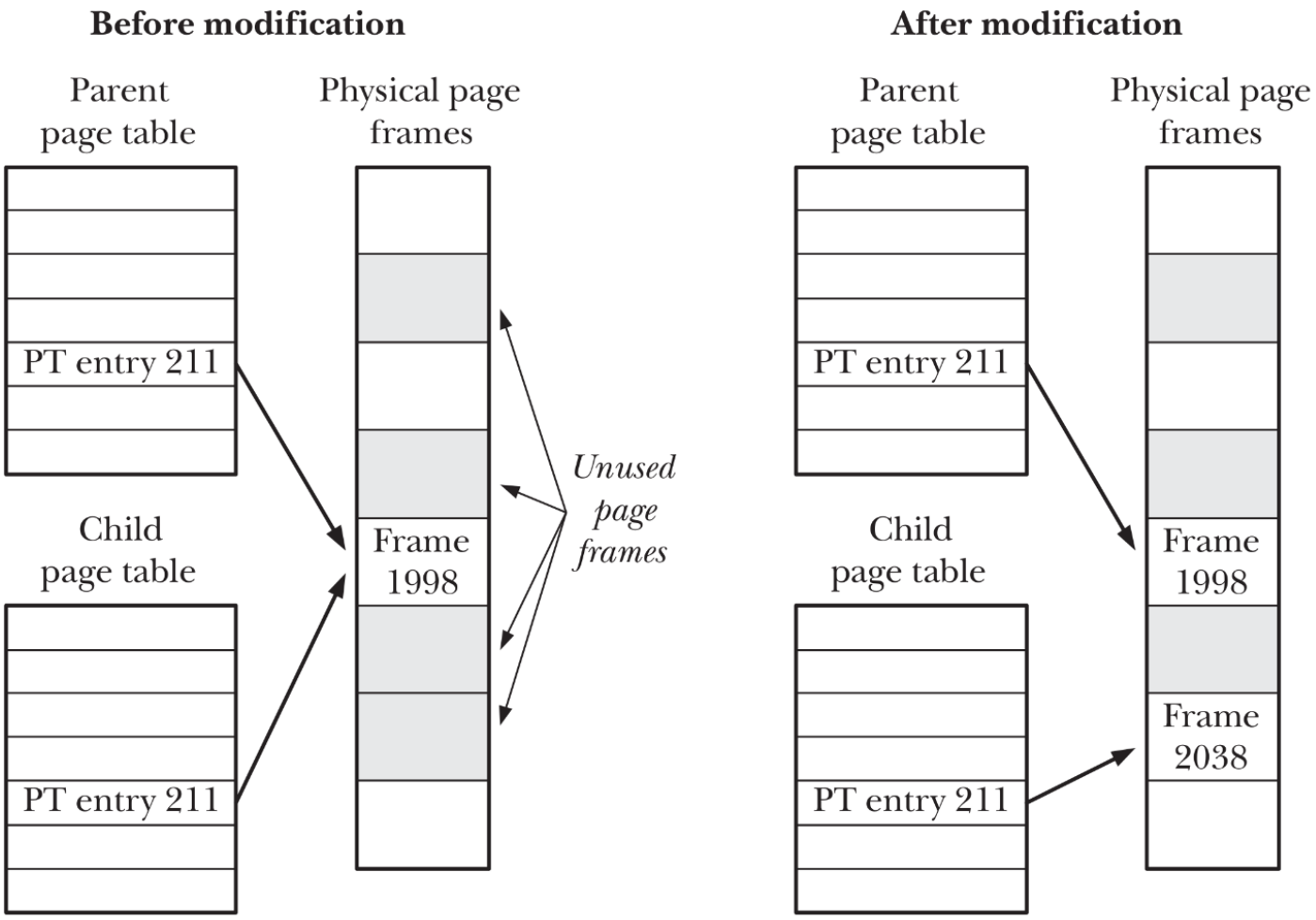
When two or more processes share the same pages, each process can potentially see the changes to the page contents made by other processes, depending on whether the mapping is private or shared:

- **Private mapping(MAP_PRIVATE)**: Modifications to the contents of the mapping are not visible to other processes. The kernel accomplishes changing contents using the copy-on-write technique(**)
- **Shared mapping (MAP_SHARED)**: Modifications to the contents of the mapping are visible to other processes

Conclution, this table write the purpose of this types:

Visiblility of modifications	File	Anonymous
Private	Initializing memory from contents of file	Memory allocation
Shared	Memory-mapped I/O; sharing memory between processes (IPC)	Sharing memory between processes (IPC)

(**): **Copy-on-write(COW)** technique: this method help kernel don't allocate new memory when processes just read, when any processes change the data of this memory, kernel will create new memory areas for this processes.



> We use file mapping when we need communicate between processes without ancestor relationship. Anonymous mapping is used for processes having parent-child relationship(creating by `fork`)

```
/* process 1 */
int main()
{
    int fd = open("/home/khanhnp/C_programming/map.txt", O_RDWR | O_CREAT, 0666);

    int *counter = 0;
    ftruncate(fd, 100);
    counter = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    while (1)
    {
        printf("%d\n", *counter);
        sleep(1);
    }
```

```
}

/* process 2 */
int main()
{
    int fd = open("/home/khanhnp/C_programming/map.txt", O_RDWR);
    int *counter = NULL;
    counter = mmap(NULL, 10000000, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    while (1)
    {
        (*counter)++;
        printf("%d\n", *counter);
        sleep(1);
    }
}
```

As we can seen in the example above, 1 process increase the value of integer number stored in shared memory, 1 process print this value.

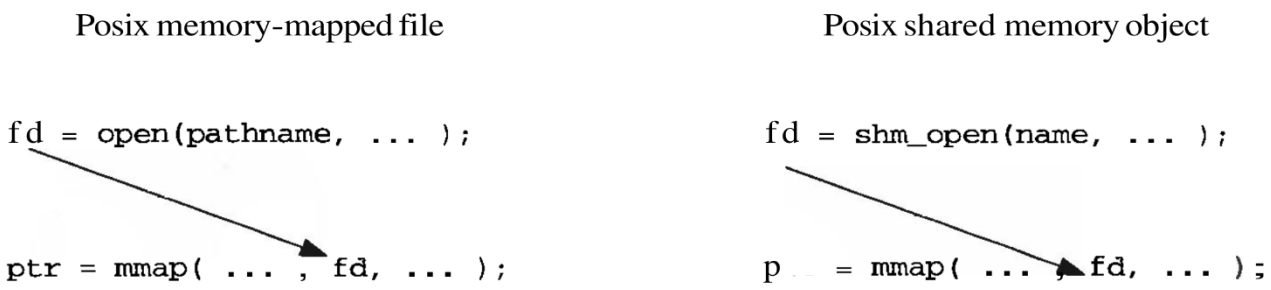
```
0
0
1 /* Process 2 started */
2
3
4
```

We can also unmapping a mapped region by `munmap()` function and resize by using `mremap()`.

> The advantage of using `mmap()` function is speed, that mean we don't need to call system call function like `read`, `write`, `lseek` between communicating process. With mmap, we just call only 1 system call 1 time, after that we can interact directly with virtual addess without calling to kernel. We can use this idea for next IPC: **shared memory**

POSIX Shared memory

Like the example above, Instead of creating a regular file by `open()` for shared memory mapping, we can create shared memory object by using `shm_open()` in POSIX shared memory.



Shared memory object have some features:

- The pathname object will be stored in temporary file system(tmpfs), not on disk like regular file, so when kernel reboot, this files will be disappeared
- We can use `shm_unlink` to remove a POSIX shared memory object pathname, the object in kernel will be released after no process have fd point to this object.

Let's do an example above by using shared memory object

```
/* process 1 */
int main()
{
    int fd = shm_open("mymem", O_RDWR | O_CREAT, 0666);

    int *counter = 0;
    ftruncate(fd, 100);
    counter = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    printf("%d\n", *counter);
    getchar();
    printf("%d", *counter);
    shm_unlink("mymem");
}

/* process 2 */
int main()
{
    int fd = shm_open("mymem", O_RDWR, 0666);
    int *counter = NULL;
    counter = mmap(NULL, 10000000, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    (*counter) ++;
    printf("process2: %d\n", *counter);
}
```

After running process 1, we can check the shared memory object pathname in `/dev/shm` like this:

```
0


1   /* process 2 started */
```

This table will list all API when using POSIX shared memory:

API	Detail
shm_open()	create a new or opens an existing shared memory object
shm_unlink()	remove the shared memory object pathname
ftruncate()	changing the size of shared memory object

Conclution, to use posix shared memory, we need 2 steps:

- Use the `shm_open()` function to create or open an existing object with a specified name. `shm_open()` returns a file descriptor referring to the object.
- Using `ftruncate()` to expand or shrink the shared memory object as desired
- Pass the file descriptor obtained in the previous step in a call to `mmap()` that specifies MAP_SHARED in the flags argument.

Problem when using shared memory and pointer

When using shared memory, we should avoid using pointers as elements of a shared structure. Each process has its own virtual address space, so a pointer stored by one process may not be valid or point to the same memory location when accessed by another process.

For example:

```
/* process 1 */
struct Data
{
    int value;
    int *ptr;
};

int main()
{
    int fd = shm_open("/mymem", O_CREAT | O_RDWR, 0666);
```

```
    ftruncate(fd, sizeof(struct Data));

    struct Data *data = mmap(NULL, sizeof(struct Data),
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED, fd, 0);

    int x = 500;

    data->value = 100;
    data->ptr = &x;

    printf("P1 data      = %p\n", data);
    printf("P1 &ptr      = %p\n", &data->ptr);
    printf("P1 ptr       = %p\n", data->ptr);
    printf("P1 *ptr      = %d\n", *data->ptr);

    getchar();

    munmap(data, sizeof(struct Data));
    shm_unlink("/mymem");
    close(fd);
}

/* process 2 */
struct Data
{
    int value;
    int *ptr;
};

int main()
{
    int fd = shm_open("/mymem", O_RDWR, 0666);

    struct Data *data = mmap(NULL, sizeof(struct Data),
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED, fd, 0);
    printf("P2 data      = %p\n", data);
    printf("P2 &ptr      = %p\n", &data->ptr);
    printf("P2 ptr       = %p\n", data->ptr);
```

```
    printf("P2 value     = %d\n", data->value);
    printf("P2 *ptr      = %d\n", *data->ptr);

    munmap(data, sizeof(struct Data));
    close(fd);
}
```

The result is:

```
P1 data      = 0x75d981763000
P1 &ptr      = 0x75d981763008
P1 ptr       = 0x7ffcf24e93a8
P1 Value     = 100
P1 *ptr      = 500


P2 data      = 0x70e0bb622000
P2 &ptr      = 0x70e0bb622008
P2 ptr       = 0x7ffcf24e93a8
P2 value     = 100
Segmentation fault (core dumped)
```

As we can see, because the pointer `ptr` stored the meaningful address value with process 1, but invalid with process 2.

> As we know from the last chapter, when more objects like threads or processes interact with same memory space, the race condition is occured. Therefore, we need synchronization methods. In process scope, we have **semaphore** and **file-lock**

POSIX Semaphore

Overall, POSIX semaphore have the definition, action like semaphore in mutex(learn from the previous chapter). The different is the scope of this semaphore is process and there are 2 types of POSIX semaphore: **Named and Unamed semaphore**

Named semaphore

Named semaphore is the semaphore that processes can access by a specific file pathname.

This table list all API in named semaphore

API	Detail
sem_open()	opens or creates a semaphore
sem_post(sem)	increment a semaphore’s value
sem_wait(sem)	decrement a semaphore’s value
sem_getvalue()	retry a semaphore’s current value
sem_close()	removes the calling process’s association with a semaphore that it previously opened
sem_unlink()	marks the semaphore for deletion when all processes have closed it

This example have race condition between process 1 and 2:

```
/* process 1 */
int main()
{
    int fd = shm_open("/mymem", O_RDWR | O_CREAT, 0666);
    int *counter;
    ftruncate(fd, 100);
    counter = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    *counter = 0;
    for (int i = 0; i < 100000; i++)
    {
        (*counter)++;
        sleep(0.5);
        printf("process 1: %d\n", (*counter));
    }
    munmap(counter, sizeof(int));
    close(fd);
    shm_unlink("mymem");
}

/* process 2*/
int main()
{
    int fd = shm_open("mymem", O_RDWR, 0666);
    int *counter = NULL;
```

```
    counter = mmap(NULL, 10000000, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    for (int i = 0; i < 100000; i++)
    {
        (*counter)++;
        sleep(0.5);
        printf("process 2: %d\n", *counter);
    }
    munmap(counter, sizeof(int));
    close(fd);
}
```

After executing process 1 first and executing process 2 when process process 1 still run. The result of process 2 when completing is :

```
process 2: 199994
process 2: 199995
```

> Let's fix by using name binary semaphore as a mutex:

```
/* process 1 */
int main()
{
    int fd = shm_open("/mymem", O_RDWR | O_CREAT, 0666);
    sem_t *sem = sem_open("/sem", O_RDWR | O_CREAT, 0666, 1);
    int *counter;
    ftruncate(fd, 100);
    counter = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    *counter = 0;
    for (int i = 0; i < 100000; i++)
    {
        sem_wait(sem);
        (*counter)++;
        sleep(0.5);
        printf("process 1: %d\n", (*counter));
        sem_post(sem);
    }
    sem_unlink("/sem");
    munmap(counter, sizeof(int));
```

```
    close(fd);
    shm_unlink("mymem");
}

/* process 2*/
int main()
{
    int fd = shm_open("/mymem", O_RDWR, 0666);
    sem_t *sem = sem_open("/sem", 0);
    int *counter = NULL;
    counter = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    for (int i = 0; i < 100000; i++)
    {
        sem_wait(sem);
        (*counter)++;
        sleep(0.5);
        printf("process 2: %d\n", (*counter));
        sem_post(sem);
    }
    sem_unlink("/sem");
    munmap(counter, sizeof(int));
    shm_unlink("/mymem");
    close(fd);
}
```

Now the result of process 2 when complete is:

```
process 2: 199998
process 2: 199999
process 2: 200000
```

Unamed semaphore

Unamed semaphore have some API same Name semaphore like sem_post, sem_wait, sem_getvalue, except:

API	Detail
shm_init()	function initializes a semaphore and informs the system of whether the semaphore will be shared between processes or between the threads of a single process

API	Detail
shm_destroy()	destroy a semaphore

The different is this semaphore don't have any pathname in file pathname. As a result, unamed semaphore only use in multithread or processes have ancestor relationship.

Other Synchronizations

We can also use others like mutex, condition variable for process synchronization by setting `PTHREAD_PROCESS_SHARED` attribute

Moreover, we know most process often interact with shared data from a shared file in file system. Therefore, we can use some technique like `file lock` or `record lock` to lock this file, avoid interacting of other processes.

File lock

File lock is the situation we call systemcall to kernel and request exclusive permisstion with this file. That mean no other processes can access this file before the exclusive process unlocking.

We can do that by using `flock()` systemcall, the argument:

Value	Detail
LOCK_EX	Place an exclusive lock on the file referred to by fd
LOCK_SH	Place a shared lock on the file referred to by fd
LOCK_UN	Unlock the file referred to by fd
LOCK_NB	Make a nonblocking lock request

Example for the example above, using file lock:

```
/* process 1 */
int main()
{
    int fd = shm_open("/mymem", O_RDWR | O_CREAT, 0666);
    int *counter;
    ftruncate(fd, 100);
    counter = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    *counter = 0;
    for (int i = 0; i < 100000; i++)
```

```
    {
        flock(fd, LOCK_EX);
        (*counter)++;
        sleep(0.5);
        printf("process 1: %d\n", (*counter));
        flock(fd, LOCK_UN);
    }
    munmap(counter, sizeof(int));
    close(fd);
    shm_unlink("mymem");
}

/* process 2 */
int main()
{
    int fd = shm_open("/mymem", O_RDWR, 0666);
    int *counter = NULL;
    counter = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    for (int i = 0; i < 100000; i++)
    {
        flock(fd,LOCK_EX);
        (*counter)++;
        sleep(0.5);
        printf("process 2: %d\n", (*counter));
        flock(fd,LOCK_UN);
    }
    munmap(counter, sizeof(int));
    shm_unlink("/mymem");
    close(fd);
}
```

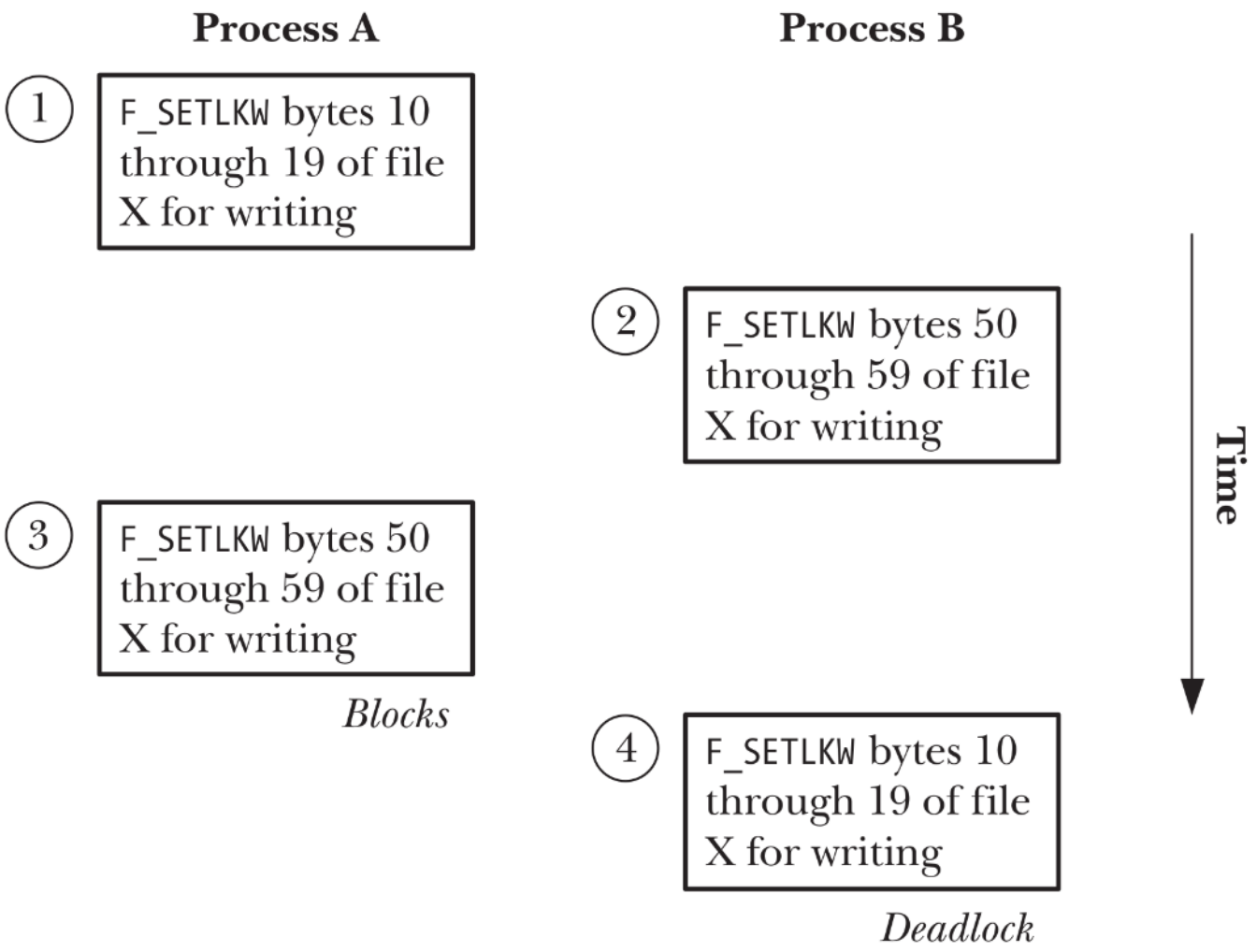
We can also lock a section of file by using `record lock` instead of locking all file:

Record lock

With record lock, we can place a lock on any part of a file, ranging from a single byte to the entire file.

> fcntl() is used to lock byte ranges corresponding to the application defined record boundaries within the file.

Be careful when using fcntl systemcall, it can make the deadlock situation like this:



Now we understand about POSIX standard IPC in linux. Next step we will go to an other standard: `System V`

System V

There are 3 types of system V IPC:

- System V message queue
- System V semaphore
- System V shared memory

In Posix IPC, processes use pathname to map with IPC object in kernel. However, in System V IPC, processes must use IPC key to link with IPC object.

To create IPC key, we must use ftok() system call with existing pathname and an integer indentifier(id).

For example, when we call this, both 2 processes will have IPC key link to same object in kernel:

```
/* process 1 */
IPC_key1 = ftok("/name",12);

/* process 2 */
IPC_key2 = ftok("/name",12);
```

> After creating IPC key, we have the way to go to IPC object in kernel. However, we don't have anything to directly interact with IPC object, so we need a non-negative integer number(identifier) to interact with IPC object.

Identifier

Each IPC structure (message queue, semaphore, or shared memory segment) in the kernel is referred to by a non-negative integer `identifier`

We can get `identifier` by using IPC key and get function: `msgget()`, `semget()`, and `shmget()`

System V message queue (MQ)

System V MQ is quite same with POSIX MQ, except:

System V MQ are identified by a **message queue identifier**.

System V MQ don't have priority field, it has the type field. Therefore, when receivers process get the message, they must get with specific type.

System V MQ have API is different from POSIX MQ. I will list in the table bellow

API	Detail
msgget()	creates a new message queue or obtains the identifier of an existing queue.
msgsnd()	writes a message to a message queue

API	Detail
msgrcv()	reads (and removes) a message from a message queue
msgctl()	performs control operations on the message queue identified

```
/* 1. Define message structure (The first field MUST be long mtype) */
struct msgbuf {
    long mtype;
    char mtext[256];
};

int main() {
    key_t key;
    int msgid;
    struct msgbuf send_msg, rcv_msg;

    /* 2. Generate a unique key (ftok) */
    key = ftok("/tmp", 'A');

    /* 3. Create or connect to the Message Queue (msgget) */
    msgid = msgget(key, 0666 | IPC_CREAT);

    /* 4. Send a message (msgsnd) */
    send_msg.mtype = 1; /* Set message type to 1 */
    strcpy(send_msg.mtext, "Hello, System V MQ!");

    /* Size is the size of the data part (mtext), NOT the whole struct */
    msgsnd(msgid, &send_msg, sizeof(send_msg.mtext), 0);

    /* 5. Receive a message (msgrcv) */
    /* mtype = 1 (receive message with mtype 1), flag = 0 (blocking) */
    msgrcv(msgid, &rcv_msg, sizeof(rcv_msg.mtext), 1, 0);

    /* 6. Remove the Message Queue (msgctl) */
    msgctl(msgid, IPC_RMID, NULL);

    return 0;
}
```

System V Shared memory

System V shared memory is similar in concept to Posix shared memory. Instead of calling `shm_open` followed by `mmap`, we call `shmget` followed by `shmat`.

System V API:

API	Detail
shmget()	create a new shared memory segment or obtain the identifier of an existing segment
shmat()	attach the shared memory segment; that is, make the segment part of the virtual memory of the calling process.
shmdt()	detach the shared memory segment
shmctl()	performs a range of control operations. We can delete the shared memory segment. The segment will be destroyed only after all currently attached processes have detached it

```
int main() {
    key_t key;
    int shmid;
    char *shm_ptr;

    /* 1. Generate a unique key (ftok) */
    key = ftok("/tmp", 'B');

    /* 2. Create or connect to the Shared Memory segment (shmget) */
    /* 1024 is the size of the shared memory in bytes */
    shmid = shmget(key, 1024, 0666 | IPC_CREAT);

    /* 3. Attach the Shared Memory segment to the process address space (shmat) */
    /* NULL lets the OS choose the address, 0 means read/write access */
    shm_ptr = (char *)shmat(shmid, NULL, 0);

    /* 4. Write data to the Shared Memory */
    strcpy(shm_ptr, "Hello, System V Shared Memory!");

    /* 5. Read data from the Shared Memory */
    printf("Read from SHM: %s\n", shm_ptr);
```

```
    /* 6. Detach the Shared Memory segment from the process (shmdt) */
    shmdt(shm_ptr);

    /* 7. Remove the Shared Memory segment (shmctl) */
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}
```

System V Semaphore

System V semaphore is similar in concept to Posix semphore, execpt API functions:

API	Detail
semget()	create a new semaphore set or obtains the identifier of an existing
semctl()	performs a variety of control operations
semop()	derforms one or more operations on the semaphores, use this function to increase or decrease semaphore counter
shmctl()	performs various control operations on a semaphore. We can use this function to remove the semaphore

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

/* Union required for semctl operations */
union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

int main() {
    key_t key;
    int semid;
    union semun arg;
```

```
    struct sembuf sb;

    /* 1. Generate a unique key (ftok) */
    key = ftok("/tmp", 'C');

    /* 2. Create or connect to the Semaphore set (semget) */
    /* 1 is the number of semaphores in the set */
    semid = semget(key, 1, 0666 | IPC_CREAT);

    /* 3. Initialize the semaphore value to 1 (semctl) */
    /* 0 is the index of the semaphore in the set, SETVAL sets the value */
    arg.val = 1;
    semctl(semid, 0, SETVAL, arg);

    /* 4. Perform P (Wait/Lock) operation (semop) */
    sb.sem_num = 0;       /* Semaphore index */
    sb.sem_op = -1;       /* Decrement by 1 (Wait) */
    sb.sem_flg = 0;       /* Blocking operation */
    /* 1 is the number of operations in the array */
    semop(semid, &sb, 1);

    /* === CRITICAL SECTION HERE === */

    /* 5. Perform V (Signal/Unlock) operation (semop) */
    sb.sem_op = 1;        /* Increment by 1 (Signal) */
    semop(semid, &sb, 1);

    /* 6. Remove the Semaphore set (semctl) */
    semctl(semid, 0, IPC_RMID);

    return 0;
}
```

Now, after understanding about some ways processes can communicate together, I will write about the special IPC. When using others IPC, processes use data and shared memory to communicate. However, this method doesn't do that, it uses a signal.

Signal

A signal is a notification to a process that an event has occurred. One process can (if it has suitable permissions) send a signal to another process.

Therefore, signals can be employed as a synchronization technique, or even as a primitive form of interprocess communication (IPC)

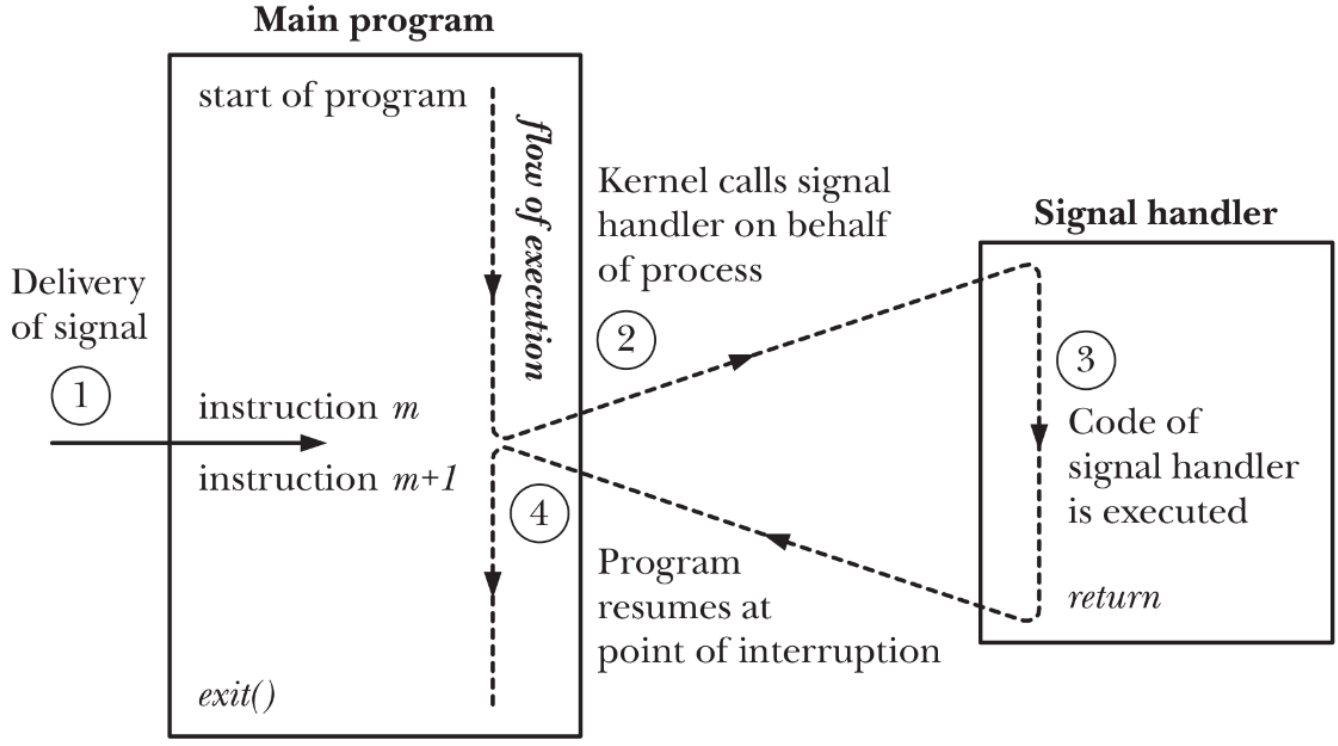
When receiving a signal, a process carries out one of the following default actions, depending on the signal:

- The signal is **ignored**; that is, it is discarded by the kernel and has no effect on the process.
- The process is **terminated** (killed). This is sometimes referred to as abnormal process termination, as opposed to the normal process termination that occurs when a process terminates using exit()
- A **core dump file** is generated, and the process is terminated
- The process is **stopped**—execution of the process is suspended
- Execution of the process is **resumed** after previously being stopped.

The man 7(signal) list 30 standard signals, so I will not list again on this document

Signal system call

`Signal()` is the system call to change the action of process when receive a signal(Signal disposition). I call this action is **signal handler** or **handle function**



For example:

```
void handle()
{
    printf("pressed CTRL + C\n");
}

int main()
{
    signal(SIGINT, handle);
    while (1)
    {
        printf("process running\n");
        sleep(1);
    }
}
```

This program makes the action triggered when the process receives a SIGINT signal (CTRL + C) a normal handler instead of terminating the program by default.

The result:

```
process running
process running
^Cpressed CTRL + C
process running
process running
process running
^Cpressed CTRL + C
process running
^Cpressed CTRL + C
process running
^Cpressed CTRL + C
process running
process running
```

> The signals SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.

> When calling fork function, the child process also have the parent's disposition

Signal problem and sigaction function

- In earlier versions of the UNIX System, the action for a signal was reset to its default each time the signal occurred. Therefore, if process receive a changed signal after the handle reseting and before changing again to the user's handle, the process will act default action of this signal. We call this situation is **Unreliable signal**
- Besides, when handling a handle of a signal, if process receive an other signal which make the process is killed, the handle action will not complete.
- Finally, when system do a "slow" system call like `read`, `write`, `sleep`, if the prcess receive any signal, this systemcalls will return error code without continuing after handle signal completing.

To solve this problems, we can use the `sigaction` function:

The sigaction() system call is used to change the action taken by a process on receipt of a specific signal

```
int sigaction(int signum, const struct sigaction *_Nullable restrict act, struct sigaction
*_Nullable restrict oldact)
```

For example:

```
void handle(int sig)
{
    for (int i = 0; i < 5; i++)
    {
        printf("pressed CTRL + C\n");
        sleep(1);
    }
}

int main()
{
    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGTERM);
    struct sigaction *act = malloc(sizeof(struct sigaction));
    act->sa_handler = handle;
    act->sa_mask = set;
    sigaction(SIGINT, act, NULL);
    while (1)
```

```
    {
        printf("process running\n");
        sleep(1);
    }
}
```

This process will avoid and push the SIGTERM signal to signal queue when the handle function of SIGINT is running.

So, After we known the way to create handle functions, I will list some rule that we must follow:

- We should only use some Reentrant function(async-signal safe) inside handle function. By using this funcions, we will avoid destroying the process memory space.
- Use `sigaction()` instead of `signal()` to have more ability controlling the handle function

Besides the sending signals from keyboard, terminal, we can use other processes to send signal by using `kill` and `raise` function

Kill and Raise function

Kill is sytem call to send a signal to other processes.

```
int kill(pid_t pid, int sig);
```

We can choose sending the signal to one ore more processes by the value of pid argument.

> One trick is if we want to check Does an other process with specific PID exist, we can use the `kill` function with `sig` argument is NULL. We can know from the error code which `kill` return

Besides, `raise` system call will help the process can send a signal to it's self.

```
int raise(int sig);
```

Moreover, we can use other helpful functions bellow to control the signal on process

Pause and Alarm function

pause() suspends execution of the process until receiving a signal

```
int pause(void);
```

`Alarm()` will set an alarm oclock, after that, kernel will send SIGALARM to this process. If we don't write the signal handler to receive this signal, the process kill be terminated.

```
unsigned int alarm(int seconds);
```

After writing about some crucial function in signal, I will write about an important knowledge: **Blocking signal**

Signal set

To presest a list of signals that we need interact, we use the variable with datatype: **signal set**

When using signal set, we have some functions that we need to know:

```
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signo);
int sigdelset(sigset_t *set, int signo);
int sigismember(const sigset_t *set, int signo);
int sigisemptyset(const sigset_t *set);
```

- sigemptyset() function initializes a signal set to contain no members
- sigfillset() function initializes a set to contain all signals
- sigaddset() function add an signal to a set
- sigdelset() function delete an signal in a set
- sigismember() function is used to test for membership of a set
- sigisemptyset() function returns true if set contains no signals.

Now, we known the way to list all signals that we needs to block or unblock. The function bellow will help us block all signals in this list:

Sigprocmask function

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

We can use `sigprocmask()` to change the process signal mask, to retrieve the existing mask, or both. The `how` argument determines the changes that sigprocmask() makes to the signal mask:

- SIG_BLOCK: Block all signal in the set
- SIG_UNBLOCK: Unblock all signal in the set

- SIG_SETMASK: Replace all signals of oldset by signals of set

> The question is where were all signals blocked by `sigprocmask` function?

To answer this question, we can go to the next section: `Pending signals`

Pending signal

If a process receives a signal that it is currently blocking, that signal is added to the process's set of pending signals

To determine which signals are pending for a pro- cess, we can call `sigpending()`:

```
int sigpending(sigset_t *set);
```

The sigpending() system call returns the set of signals that are pending for the calling process in the sigset_t structure pointed to by set

> One problem with pending signal is: if we send further instances of a standard signal that is already pending for a process, that signal is delivered only once.

> > If we want to process can receive full signal in pending signal, we can use realtime signal

Realtime signal

Realtime signal have some features:

- Realtime signals provide an increased range of signals that can be used for application-defined purposes
- If multiple different realtime signals are pending, then the highest-numbered signal is delivered first
- When sending a realtime signal, it is possible to specify data
- If multiple instances of a realtime signal are sent to a process, then the signal is delivered multiple times

>

> We can call the realtime signal by `SIGRTMIN` and `SIGRTMAX` with the `+` operator

sigqueue() system call sends the realtime signal specified by sig to the process specified by pid

```
int sigqueue(pid_t pid, int sig, const union sigval value);
```

We can use the flag `SA_SIGINFO` to receive the value data in function `sigaction()`

Sigsuspend function

Imagine we want unblock the signal and pause to sleep before receiving any signal. However, in the time between the unblock and pause function, the signal is sent. As a result, the process will handle this signal before sleeping and never wake up again.

> To fix this problem, we need make this action become atomic. `sigsuspend` function can do that.

```
int sigsuspend(const sigset_t *sigmask);
```

When we call the `sigsuspend` function, that mean we atomically call `unblock` and `pause` function to this process

Sumary

In this document, I write about many system call function. This is too difficult to remember them in short time. In this part, I will list all api in of IPC process, so we can easily check again:

Lab practice

- **Lab 1: Chat program using FIFO**

https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/IPC/chat_program

- **Producer-consumer using Shared memory and semaphore**

https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/IPC/producer_consumer

---

Revision #9
Created 2026-09-07 03:23:49 UTC by Khanh Pham
Updated 2026-09-22 09:48:26 UTC by Khanh Pham